

Agent Harness

Design and implementation report — an LLM ↔ tool loop for an operations assistant

Version	Date	Author	Changes	Status
1.0	2026-09-30	Dang Hoang Long	Initial version	Final

Summary

Implemented and documented with Claude Code from a design spec I wrote; architecture, safety design and trade-off decisions are my own.

Approach. A hand-written loop between an LLM and three mock tools (knowledge-base search, service status, incident creation). A run is persisted and advanced one step at a time; it stops at a final answer or a limit, and pauses whenever the model wants to create an incident, until a human approves or rejects that exact call. Every decision is recorded as a trace event. The harness is reachable through a REST API and a CLI, and runs without an API key using a scripted model.

Five design decisions.

1. **Re-entrant loop over persisted state.** `advance()` reads the run, works until it finishes or needs a decision, saves and returns; a later call resumes it. Waiting for a human is a normal return, not a blocked coroutine (§2).
2. **Approval gate in code, scoped to one call.** Dispatch never runs an approval-gated tool; it parks the run on an approval that stores the validated arguments, and exactly those arguments run after approval. A rejected call is refused if proposed again (§6).
3. **Retry only transient failures, in one place each.** Tool calls retry in the executor, LLM calls in the loop, both with capped exponential backoff and full jitter; the SDK does not retry (§5).
4. **Idempotent side effect.** `create_incident` runs with a harness-generated key the model never sees; a retry after a post-commit timeout returns the original incident (§5).
5. **Errors are context; execution is bounded.** Tool failures reach the model as structured observations; limits on steps, active time, repeated calls and consecutive errors end runs that do not converge, and time spent waiting for approval is not counted (§3, §6).

How to run. `make install`, then `make demo` (scripted scenario, you approve the incident) or `make serve` for the API with `docs/postman_collection.json`. `make check` runs lint, types and 254 tests. A real model is any OpenAI-compatible endpoint, e.g. Ollama with `qwen2.5:7b` (§12).

Live run. With `qwen2.5:7b` on Ollama, 2 of 5 scenarios followed the expected tool sequence; the others drifted off task, and one final answer claimed an incident that was never created. The harness handled every run correctly, and no incident was created without an approval (§8.2, §11.1).

Main limitations. Single process with SQLite; a tick is not atomic across a crash and there is no resume-after-crash; approvals do not expire and approvers are not authenticated; cancellation is cooperative; one tool call per step; final answers are not checked against the trace; the prompt-injection defence is basic, with the approval gate limiting the consequence (§13).

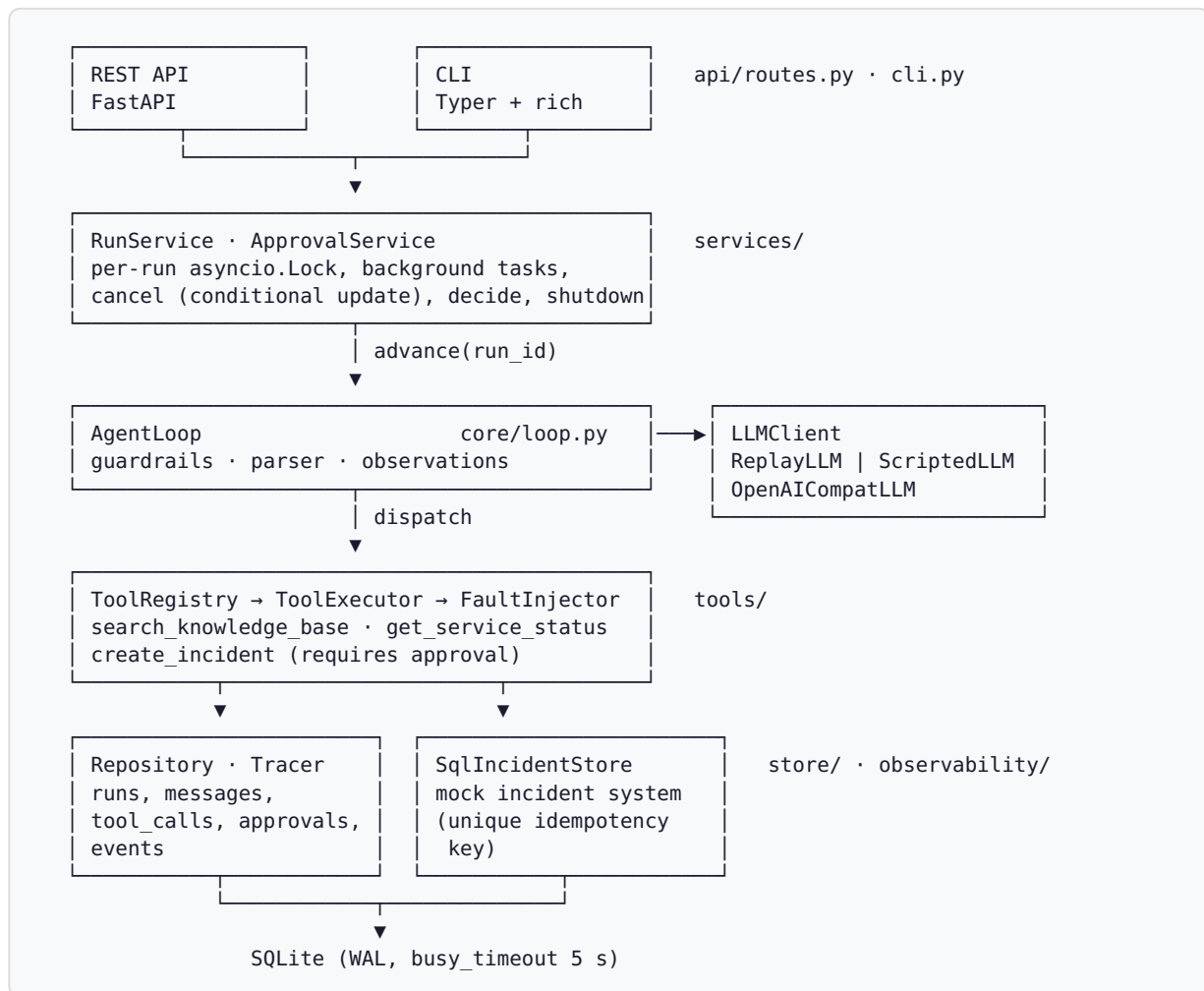
1. Overview and goals

A user gives an objective (for example “customers report failed checkouts; investigate and open an incident if warranted”). The harness runs a hand-written loop between an LLM and three mock tools until the model gives a final answer or a stopping condition is reached. Every step is persisted, every decision point is traced, and the one side-effecting tool, `create_incident`, only runs after a human approves the exact call.

Requirement	How it is met
LLM ↔ tool loop	<code>AgentLoop.advance()</code> in <code>core/loop.py</code> ; no agent framework
Input / output validation	Pydantic input and output model per tool; arguments validated in dispatch, results in the executor
State and history	SQLite via SQLAlchemy async: runs, messages, tool calls, approvals, events
Failure handling	Error taxonomy with retryable vs. non-retryable classes; one retry layer each for tools and LLM
Execution limits	Step, active-time, repeated-call and consecutive-error limits
Human approval	Approval gate in dispatch; the stored, approved arguments are what executes
Trace	Append-only <code>events</code> table plus structlog JSON lines; <code>GET /runs/{id}/trace</code> , <code>harness trace</code>
Interfaces	REST API (FastAPI) and CLI (Typer); Postman collection; zero-setup <code>make demo</code>

Scope is deliberately limited to the baseline plus a few production details (PLAN §2). Out-of-scope items are listed in §13 and §14.

2. Architecture



2.1 Components and responsibilities

The REST API and the CLI are thin: they validate input and call two services. `RunService` creates runs, advances them one at a time per run (a per-run `asyncio.Lock`), runs them in background tasks, cancels them, and fails runs that are still in flight when the process shuts down. `ApprovalService` records a human decision and schedules the run to resume. Both the API and the CLI use the same services, so an approval behaves the same wherever it is made.

`AgentLoop` in `core/loop.py` is the whole agent loop (about 630 lines). It delegates three small pure pieces: `parser` (what did the model ask for), `guardrails` (may the run continue) and `observations` (what the model sees after a tool call). Tools sit behind a registry and an executor that owns timeouts, retries and output validation. The repository returns pydantic records and opens one short transaction per call, so no database session is held across an LLM or tool call. `wiring.py` is the composition root: the one place that turns settings into this object graph, for the API, the CLI and the tests.

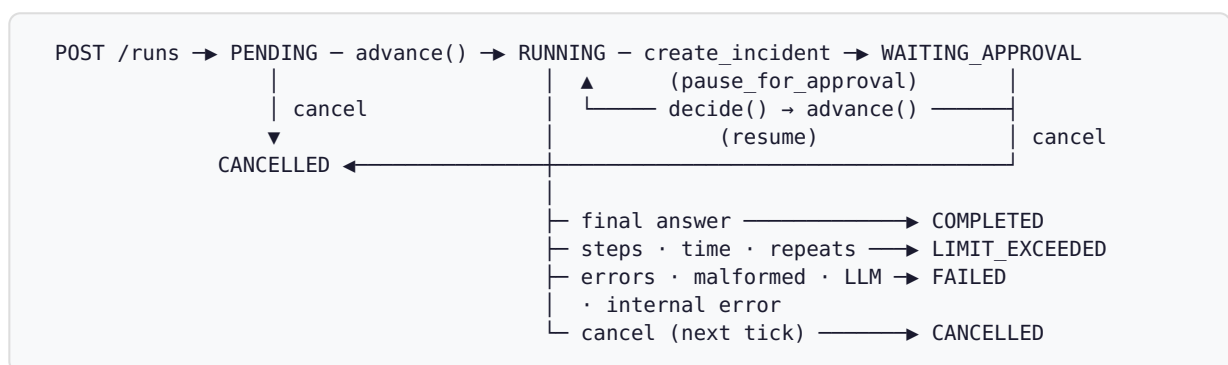
2.2 Why a re-entrant loop over persisted state (P2)

A run has to stop and wait for a human, possibly for hours, and continue when the decision arrives in a different HTTP request. There are two ways to do that. The loop can stay suspended in memory, awaiting a future that the decision resolves. Or the loop can be re-entrant: each call reads the run from the database, works until the run finishes or needs a decision, saves, and returns. The harness does the second [R5].

- **Pausing is a normal return.** `Repository.pause_for_approval` writes the approval, the run's `pending_tool_call` and the `WAITING_APPROVAL` status in one transaction, so a decision can never find a pending approval on a run that is not yet waiting for it.
- **Resuming is any later `advance()`** whose run has a decided approval (`_ready_to_advance`). A decision that arrives while an earlier `advance()` still holds the run lock is picked up by the next one; a test holds the lock with an `asyncio.Event` to prove it.
- **Behaviour is fixed per run.** The effective limits and faults are snapshotted on the run (`RunConfig`), so a run resumed after a restart keeps the limits it started with.
- **Waiting is not working.** The run's clock only runs inside `advance()` (`_charge_active_time`), so time waiting for a human never counts against `MAX_RUN_SECONDS`.

The cost is that every tick reads and writes the database, and anything not saved before `advance()` returns is gone. For a single process the in-memory approach would be simpler, but it loses the run on restart and ties the decision to the process that started the run. The re-entrant loop also makes testing direct: a test drives a run to the pause, makes a decision, and drives it again.

3. Run lifecycle and the tick



Termination reason	Status	Trigger (default)
<code>final_answer</code>	COMPLETED	Text reply with no tool calls
<code>max_steps</code>	LIMIT_EXCEEDED	<code>step_count ≥ MAX_STEPS</code> (12), checked before each LLM call
<code>max_run_time</code>	LIMIT_EXCEEDED	<code>active_elapsed_ms ≥ MAX_RUN_SECONDS</code> (120 s of active time)

<code>repeated_tool_call</code>	LIMIT_EXCEEDED	Same tool + canonical args already made <code>REPEAT_CALL_LIMIT</code> (3) times
<code>too_many_tool_errors</code>	FAILED	<code>MAX_CONSECUTIVE_TOOL_ERRORS</code> (3) failed executions in a row
<code>malformed_llm_output</code>	FAILED	More than <code>MAX_PARSE_RETRIES</code> (2) consecutive empty replies / invalid calls
<code>llm_unavailable</code>	FAILED	Retryable LLM errors after <code>LLM_MAX_RETRIES</code> (2) retries
<code>llm_request_rejected</code>	FAILED	Non-retryable LLM error (400/401/403/404)
<code>internal_error</code>	FAILED	Unexpected exception in <code>advance()</code> , or interrupted by shutdown
<code>cancelled_by_user</code>	CANCELLED	<code>POST /runs/{id}/cancel</code>

3.1 One tick (`AgentLoop._tick`)

```

1. re-read the run; stop if CANCELLED
2. pending_tool_call? → _resolve_pending
3. guardrails.check → max_steps | max_run_time
4. _call_llm: retry timeouts and retryable errors
5. step_count += 1; parser.parse(response)
   EmptyResponse → repair message; consecutive_malformed += 1
   FinalAnswer → store answer; finish(final_answer)
   ToolCallAction → store assistant message; answer extra calls "not executed";
   _dispatch(first call):
       parse JSON → registry.get → input model validation
       repeat limit → previously rejected? → requires approval?
       yes → pause_for_approval → WAITING_APPROVAL (return)
       no → executor.run → observation + tool_calls row
6. consecutive_malformed > 2 or consecutive_tool_errors ≥ 3 → FAILED
7. charge active time; save (refused if the run was cancelled meanwhile)

```

cancel between ticks
approved: stored args, key
run_id:approval_id; rejected:
ApprovalRejected observation
P8
P5, backoff with jitter

4. Tool integration and validation

4.1 The tool contract (`tools/spec.py`)

Field	Purpose
<code>input_model</code>	Pydantic model; source of the LLM-facing JSON schema (<code>model_json_schema()</code>); <code>extra="forbid"</code>
<code>output_model</code>	Handler output is validated against it; a violation is <code>ToolOutputValidationError</code> (not retried)
<code>handler</code>	<code>async (args, ToolContext) → dict</code>
<code>timeout_s</code>	Per attempt, capped by the run's remaining active time
<code>retry</code>	<code>RetryPolicy(max_retries=2, base_delay_s=0.5, max_delay_s=4.0)</code> , full jitter

<code>requires_approval</code>	Routes the call through the approval gate in dispatch
<code>ToolContext</code>	<code>run_id</code> , <code>step</code> , <code>idempotency_key</code> (set only for approved calls), <code>deadline_remaining_s</code> ; never shown to the model

4.2 The three mock tools

Tool	Input	Output	Timeout	Notes
<code>search_knowledge_base</code>	<code>query</code> 1-500 chars, non-blank; <code>top_k</code> 1-10 (3)	<code>results[{'doc_id', 'title', 'snippet' ≤ 500, 'score'}]</code>	3 s	BM25L over 8 markdown docs; snippet from the best-matching paragraph onward
<code>get_service_status</code>	<code>service_name</code> <code>^[a-z0-9][a-z0-9-]{0,63}\$</code>	status, error rate, p95, last_checked, dependencies	3 s	Unknown service → <code>ToolPermanentError</code> listing known services
<code>create_incident</code>	<code>title</code> 10-120, <code>description</code> 20-2000, <code>severity</code> SEV1-4	<code>incident_id</code> INC-#####, status, severity, created_at, deduplicated	5 s	Requires approval; refuses without an idempotency key; unique key in the store

4.3 Executor (`tools/executor.py`)

```

for attempt in 1 .. max_retries + 1:
    async with asyncio.timeout(min(spec.timeout_s, ctx.deadline_remaining_s)):
        raw = await faults.wrap(spec)(args, ctx)
        data = spec.output_model.model_validate(raw)      # failure → not retried
        emit tool_succeeded → return ToolResult(ok)
    on ToolError e:
        retryable and attempts left → emit tool_attempt_failed, clock.sleep(backoff), retry
        else                          → emit tool_failed → return ToolResult(error)

```

4.4 Error taxonomy (`domain/errors.py`)

Class	Retryable	Raised by
<code>ToolNotFoundError</code>	no	registry lookup in dispatch
<code>ToolArgumentsError</code>	no	dispatch: invalid JSON, not an object, schema violation (with field errors)
<code>ToolTimeoutError</code>	yes	executor, on <code>asyncio.timeout</code>
<code>ToolTransientError</code>	yes	tools / flaky fault
<code>ToolPermanentError</code>	no	tools / error fault
<code>ToolOutputValidationError</code>	no	executor output check
<code>LLMTimeoutError</code>	yes	loop backstop timeout; OpenAI client timeout
<code>LLMProviderError</code>	per error	OpenAI client: connection, 408, 429, 5xx → yes; other 4xx → no

Tool errors reach the model as `{"ok": false, "error": {"type", "message", "retryable"}}`. Service errors (`ServiceError` subclasses) carry the API error code: `run_not_found`, `approval_not_found`, `approval_not_pending`, `run_not_waiting`, `run_already_terminal`, `invalid_config_overrides`.

5. Failure handling

5.1 Retry matrix

Failure	Handled in	Retry	Returned to model	Terminates run
Tool timeout / transient error	<code>ToolExecutor.run</code>	yes, ≤ 2 retries, full jitter (cap 0.5 s, 1 s, ... ≤ 4 s)	yes, if retries exhausted	after 3 consecutive failures
Tool permanent error	<code>ToolExecutor.run</code>	no	yes	after 3 consecutive failures
Tool output fails its schema	<code>ToolExecutor._validate_output</code>	no	yes	after 3 consecutive failures
Invalid JSON / schema / unknown tool	<code>AgentLoop._dispatch</code>	no	yes, with field errors	> 2 consecutive (<code>malformed_llm_output</code>)
Empty LLM reply	<code>AgentLoop._tick</code>	repair message	—	> 2 consecutive (<code>malformed_llm_output</code>)
Extra tool calls in one reply	<code>_refuse_extra_call</code>	no	"not executed" observation	no
LLM timeout / 408 / 429 / 5xx / connection	<code>AgentLoop._call_llm</code>	yes, ≤ 2 retries, same backoff	—	<code>llm_unavailable</code> when exhausted
LLM 400 / 401 / 403 / 404	<code>AgentLoop._call_llm</code>	no	—	<code>llm_request_rejected</code>
Backoff uses up active time	<code>AgentLoop._call_llm</code>	stops	—	<code>max_run_time</code>
<code>create_incident</code> times out after commit	executor + idempotency key	yes	original incident, <code>deduplicated=true</code>	no
Concurrent create with same key	<code>SqlIncidentStore.create</code>	—	winner's incident, <code>deduplicated=true</code>	no

Unexpected exception	<code>AgentLoop.advance</code>	no	—	<code>internal_error</code>
Process shutdown mid-run	<code>RunService.shutdown</code>	no	—	<code>internal_error</code> ("shutdown")

5.2 Retry only what is worth retrying, at one layer (P5)

A retry helps only when the failure is temporary. The error taxonomy makes that a property of the error class: timeouts and transient errors are retryable; bad arguments, unknown tools, permanent errors and invalid tool output are not, because the same call would fail the same way. Each kind of call is retried in exactly one place: tool calls in `ToolExecutor.run`, LLM calls in `AgentLoop._call_llm`. The OpenAI SDK client is created with `max_retries=0`, because stacked retry layers multiply attempts (three layers of three attempts is 27 calls) and hide latency [R6].

Delays use capped exponential backoff with full jitter: `uniform(0, min(4 s, 0.5 s · 2^(attempt-1)))`. Jitter keeps many callers from retrying in lockstep against a service that is recovering [R6]. Sleeps go through the injected `Clock`, so tests assert the exact delays without waiting, and the random generator is seeded from `MOCK_SEED`. Every attempt's timeout is capped by the run's remaining active time, and LLM retries stop when the backoff has used up the run's time budget (`max_run_time`).

5.3 Errors are context (P7)

A failed tool call is information the model can act on. An unknown service returns an error listing the known services; a schema violation names the fields; a timeout says it happened and that it was retryable. These reach the model as `{"ok": false, "error": {...}}` observations and the run continues. Only failures the model cannot fix end a run: the LLM is unavailable or rejects the request, a limit is hit, or the harness itself fails. Each ends with a recorded termination reason and a `run_finished` event. The consecutive-error limits stop the case where the model keeps producing failures instead of recovering.

5.4 Side-effecting retries are idempotent (P6)

Retrying `create_incident` after a timeout is only safe if a repeat cannot create a second incident: the timeout may happen after the incident was committed (the `timeout_after_commit` fault reproduces this). The harness passes an idempotency key, `{run_id}:{approval_id}`, set only in `_resolve_pending` and absent from every tool schema, so the model can neither see nor supply it [R7]. The incident store has a unique constraint on the key. The tool looks the key up first and returns the stored incident with `deduplicated=true`; if two calls race past the lookup, the loser hits the constraint, re-reads, and returns the winner's incident. The integration test sees two attempts and one incident.

6. Safety controls

6.1 OWASP Top 10 for LLM Applications 2025 mapping [R8]

Risk	Control in this harness	Code	Test
LLM01 Prompt injection	Tool output is JSON-wrapped and truncated (4000 chars); the system prompt says tool output is data; the approval gate is the hard backstop	<code>core/observations.py</code> , <code>llm/prompts.py</code>	<code>test_prompt_injection_cannot_bypass_approval</code>
LLM05 Improper output handling	Every tool call is parsed and validated against a strict pydantic model before anything runs; extra fields rejected	<code>_parse_arguments</code> , <code>_validate_arguments</code>	<code>test_tool_inputs.py</code> , <code>test_schema_violation_reports_field_errors</code>
LLM06 Excessive agency	Approval gate in dispatch; stored args executed; one approval per call; rejected call refused in code; tool refuses without key	<code>_dispatch</code> , <code>_resolve_pending</code> , <code>_refuse_rejected_call</code> , <code>tools/incidents.py</code>	<code>test_approve_executes_stored_args_exactly_once</code> , <code>test_rejected_call_is_refused_without_new_approval</code>
LLM10 Unbounded consumption	Step, active-time, repeated-call and error limits; per-attempt timeouts; overrides may only lower limits; hard caps 50 steps / 600 s	<code>core/guardrails.py</code> , <code>config.build_run_config</code>	<code>test_max_steps_exceeded</code> , <code>test_max_run_time_exceeded</code> , <code>test_repeated_identical_call_terminates</code>

6.2 Safety is enforced in code, not in the prompt (P3)

A system prompt can ask the model to wait for approval; it cannot make it. The approval gate is therefore in dispatch: a call to a tool with `requires_approval` is never executed from dispatch. The run is paused on an approval, and the tool's handler is reached only through `_resolve_pending` , after a stored decision. A decision is one conditional update that requires the approval to be pending and its run to be waiting, so a second decision gets `approval_not_pending` and a decision after a cancel gets `run_not_waiting` . As a second line of defence, the tool refuses to run without the harness-generated idempotency key, which only the approved path provides.

A human's rejection is enforced the same way. If the model proposes the exact call again (same tool and canonical arguments, compared by `args_hash`), dispatch answers it with the earlier rejection instead of asking the human a second time.

6.3 Approve exactly what executes (P4)

An approval covers one concrete action, not a tool. The approval row stores the validated, canonical arguments and the tool call id. On approval the harness executes those stored arguments, validated again, instead of asking the model to repeat the call, which could produce arguments the human never saw. A changed call, such as a SEV2 incident proposed after a SEV1 was rejected, is a different action and waits for its own approval.

6.4 Tool output is untrusted data (P9)

Knowledge-base documents and vendor notes can contain text written to steer the model. The harness wraps every observation in a JSON envelope, caps it at 4000 characters, and the system prompt says that tool output is data, never instructions. This lowers the chance that injected text is followed; it does not prevent it. The fixture `vendor-note-untrusted.md` contains an injection line, and the test scripts a model that obeys it: the injected `create_incident` call pauses at the gate, and no incident exists unless a human approves. The real defence is that the harmful consequence needs a human.

6.5 Bounded execution (P8)

Every run is bounded: 12 steps, 120 s of active time, 3 identical calls, 3 consecutive tool errors, 2 consecutive malformed replies, and a timeout on every attempt. Active time excludes the wait for approval. The API has no authentication, so a caller may lower these limits for its own run but never raise the operator's values (`422 invalid_config_overrides`); the env values themselves are capped at 50 steps and 600 s.

7. Database design

Table	Key columns	Constraints / indexes	Written by
<code>runs</code>	id (uuid), objective, status, termination_reason, final_answer, step_count, active_elapsed_ms, consecutive_tool_errors, consecutive_malformed, pending_tool_call (json), config (json), created/updated/completed_at	PK id; index status. Updates are conditional on a non-terminal status	loop, <code>cancel_run</code>
<code>messages</code>	run_id, seq, role, content, tool_calls (json), tool_call_id, created_at	UNIQUE (run_id, seq); FK runs	loop, <code>create_run</code>
<code>tool_calls</code>	run_id, step, tool_call_id, tool_name, args (json), args_hash, status, result (json), error_type, attempts, duration_ms	INDEX (run_id, args_hash) for the repeated-call and rejected-call checks; FK runs	loop

approval s	id (uuid), run_id, tool_call_id, tool_name, args (json), status, reason, decided_by, requested_at, decided_at	FK runs; decided by a conditional UPDATE	pause_for_approval , decide_approval
events	run_id, seq, ts, type, step, payload (json)	UNIQUE (run_id, seq); append-only	Tracer.emit
incident s	id, idempotency_key, title, description, severity, status, created_at	UNIQUE idempotency_key; no FK (stands in for an external system)	SqlIncidentStore

- Timestamps come from the injected `Clock ; UTCDateTime` always returns timezone-aware UTC (SQLite has no time zones).
- JSON columns use `none_as_null` , so Python `None` is SQL NULL.
- Sequence numbers are `max(seq)+1` under a process-level lock, with the unique constraint as backstop.
- File-backed SQLite runs in WAL mode with `busy_timeout=5000` ; tests use in-memory SQLite (and a file database for concurrency tests). Only portable types are used, so the schema runs on Postgres; no migrations (`create_all` at startup).

8. Observability

`Tracer.emit(run_id, event_type, step, **payload)` appends an `events` row and logs one structlog JSON line with the same fields (payload nested under `payload`). Log values longer than 500 characters are truncated; the API key is a `SecretStr` and never rendered.

Event	Emitted by	Payload
run_created	RunService	objective, config snapshot
run_started / run_resumed	loop	—
llm_response	loop	attempt, latency_ms, usage, tool_calls, has_text
llm_retry	loop	attempt, error_type, message, delay_s
llm_malformed_output	loop	problem, consecutive
tool_invalid_call	loop	tool, error_type (arguments, not found, NotExecuted, ApprovalRejected)
tool_attempt_failed / tool_succeeded / tool_failed	executor	tool, attempt(s), error_type, delay_s / duration_ms
approval_requested / approval_decided	loop / ApprovalService	approval_id, tool, args / decision, reason, decided_by
limit_exceeded , run_finished	loop, RunService	reason, status, error details

8.1 Scripted trace: `checkout_outage` , `approve path` (`make demo`)

seq	+ms	step	event	details
1	0	0	run_created	objective, config
2	6	0	run_started	
3	10	1	llm_response	tool_calls: [search_knowledge_base]
4	14	1	tool_succeeded	search_knowledge_base, attempts 1
5	21	2	llm_response	tool_calls: [get_service_status]
6	24	2	tool_succeeded	get_service_status (checkout-api)
7	31	3	llm_response	tool_calls: [get_service_status]
8	35	3	tool_succeeded	get_service_status (payment-gateway)
9	45	4	llm_response	tool_calls: [create_incident]
10	51	4	approval_requested	create_incident, SEV1, stored args
11	57	4	approval_decided	approved by oncall@example.com
12	62	4	run_resumed	
13	67	4	tool_succeeded	create_incident → INC-000001
14	72	5	llm_response	final text
15	76	5	run_finished	completed / final_answer

8.2 Live trace: `checkout_outage` , `qwen2.5:7b` via `Ollama`

From `make test-live` on 2026-09-30 (full output in `docs/report/live_run.txt`). The model sent two tool calls per reply although the request set `parallel_tool_calls=false` . The harness executed the first call and answered the second with a “not executed” observation (events 4 and 7), so every call id still got a reply. This is the only repair the harness had to make in the live run; there were no invalid-JSON, unknown-tool or schema-violating calls.

seq	+ms	step	event	details
1	0	0	run_created	
2	23	0	run_started	
3	6274	1	llm_response	tool_calls: [get_service_status, search_knowledge_base]
4	6300	1	tool_invalid_call	search_knowledge_base: NotExecuted (second call in one reply)
5	6307	1	tool_succeeded	get_service_status (checkout-api)
6	22570	2	llm_response	tool_calls: [search_knowledge_base, create_incident]
7	22607	2	tool_invalid_call	create_incident: NotExecuted (second call in one reply)
8	22618	2	tool_succeeded	search_knowledge_base
9	52748	3	llm_response	final text

10	52762	3	run_finished	completed / final_answer
----	-------	---	--------------	--------------------------

The run is a model failure, not a harness failure. The model never proposed `create_incident` again after it was not executed, never checked `payment-gateway`, and its final answer claims it “opened an incident ... SEV3” with a placeholder id and that the dependencies were healthy. No incident exists: `create_incident` was never executed and never reached the approval gate. The trace is what shows the claim is false.

9. Technology stack

Concern	Choice	Rationale
Language, packaging	Python 3.12, uv, pyproject	Async ecosystem for LLM and HTTP; uv gives a locked, reproducible environment (<code>uv.lock</code>)
Loop	Hand-written	The loop is what is being evaluated (P1); every decision point is visible in one file
Validation, config	Pydantic v2, pydantic-settings	One mechanism for tool I/O, LLM actions, API bodies, records and env config
API, CLI	FastAPI + uvicorn; Typer + rich	Typed request validation and OpenAPI docs; readable terminal approvals and timelines
Persistence	SQLAlchemy 2.0 async +aiosqlite	Zero-setup SQLite with a schema portable to Postgres
LLM client	<code>openai</code> SDK, OpenAI-compatible endpoint	One client for OpenAI, Ollama, vLLM via <code>LLM_BASE_URL</code> ; SDK retries disabled
KB search	<code>rank-bm25</code> (BM25L)	Deterministic, no external service; BM25L because Okapi's IDF is ≤ 0 for terms in half an 8-document corpus
Logging	structlog (JSON)	Machine-readable lines with the same fields as trace events
Quality	pytest, pytest-asyncio, pytest-cov, httpx, ruff, mypy (strict)	Deterministic tests via <code>ScriptedLLM</code> and <code>FakeClock</code>
Delivery	Dockerfile, Makefile, GitHub Actions	CI runs <code>make check</code> and <code>docker build</code> ; this report renders in Docker (WeasyPrint + Pango)

10. Comparison with existing runtimes

The harness solves the same problems as two general-purpose runtimes, the OpenAI Agents SDK [R2][R3] and LangGraph [R4], in a narrower way. The table compares only what the cited pages state.

	This harness	OpenAI Agents SDK	LangGraph
Loop	Hand-written <code>advance()</code> ; one tool call per step, extra calls answered “not executed”	Runner loop: call the LLM; stop on final output, switch agent on a handoff, or run the tool calls and loop. All local function tool calls from one turn are started, with an optional concurrency cap [R2]	Application-defined graph; <code>interrupt()</code> can be called in any node [R4]
Turn limit	Steps, active time, repeated calls, consecutive errors	<code>max_turns</code> ; exceeding it raises <code>MaxTurnsExceeded</code> [R2]	Not covered in [R4]
Pause and resume	Run persisted in SQL; any later <code>advance()</code> resumes it	Run pauses with <code>interruptions</code> ; <code>RunState</code> is serialised with <code>to_json()</code> / <code>to_string()</code> and restored with <code>from_json()</code> / <code>from_string()</code> [R3]	Checkpointers plus <code>thread_id</code> ; resume with <code>Command(resume=...)</code> ; the interrupted node restarts from its beginning [R4]
Approval scope	One tool call and its stored arguments; rejection of the same call enforced	<code>needs_approval</code> (bool or per-call function); <code>approve()</code> / <code>reject()</code> per call, or <code>always_approve</code> / <code>always_reject</code> for the rest of the run [R3]	Written by the application in the node; the page shows approve, edit and reject patterns [R4]
History	<code>messages</code> table	Sessions, or <code>to_input_list()</code> [R2]	Checkpointed graph state [R4]

The shared idea is that pausing for a human must survive the request that started the run: the Agents SDK serialises a run state, LangGraph saves a checkpoint, and this harness persists the run itself. Two differences are deliberate. First, the harness binds an approval to one call and executes the stored arguments; the Agents SDK page describes approving tool calls but does not say which arguments run after approval, and it offers “always approve” for a tool for the rest of a run, which this harness does not. Second, the harness runs one tool call per step, where the Agents SDK starts all function calls from a turn; one call per step keeps a single pause point and a single pending call per run. LangGraph's rule that an interrupted node restarts from its beginning means code before `interrupt()` runs twice, which is the same reason this harness makes its only side effect idempotent (P6).

Using either runtime would have been reasonable in production. For this exercise the loop is the thing being evaluated (P1), and a hand-written loop keeps every decision point visible in one file. A router such as LiteLLM [R9] would fit behind the `LLMClient` interface for multi-

provider routing and fallbacks; its own retries would have to be disabled to keep one LLM retry layer (P5).

11. Testing strategy

Tests never call a real LLM or the network. `ScriptedLLM` replays responses, exceptions and callables; `FakeClock` makes backoff and active-time checks instant and exact; tool timeouts use real time with ≤ 50 ms values. Suites: 187 unit, 41 integration (loop + SQLite + `ScriptedLLM`), 26 API (httpx over the ASGI app), 5 optional live (skipped without `LLM_API_KEY`). Coverage on `src/harness` $\approx 98\%$ (target $\geq 85\%$, excluding `openai_compat.py` and `cli.py`).

Required category	Integration tests	Unit / API tests
Success	<code>test_run_completes_with_kb_and_status_calls</code> , <code>test_final_answer_without_tools</code>	tool handlers, BM25, parser; API create → approve → completed
Tool failure	<code>test_flaky_tool_recovers_after_retries</code> , <code>test_permanent_error_returned_to_llm_and_run_completes</code> , <code>test_tool_timeout_reported_to_llm</code> , <code>test_consecutive_tool_errors_fail_run</code> , <code>test_timeout_after_commit_does_not_duplicate_incident</code>	executor (success, 2 transient + success with 2 sleeps, exhausted, timeout, permanent, output validation), backoff bounds, faults, incident race
Malformed LLM output	<code>test_invalid_json_arguments_then_recovery</code> , <code>test_unknown_tool_then_recovery</code> , <code>test_schema_violation_reports_field_errors</code> , <code>test_repeated_empty_responses_fail_run</code> , <code>test_llm_provider_errors_then_llm_unavailable</code> , <code>test_llm_timeout_is_retried</code>	parser, OpenAI client error mapping (mock transport)
Approval	<code>test_create_incident pauses for approval</code> , <code>test_approve_executes_stored_args_exactly_once</code> , <code>test_reject_resumes_and_completes_without_incident</code> , <code>test_prompt_injection_cannot_bypass_approval</code> , <code>test_rejected_call_is_refused_without_new_approval</code> , <code>test_decision_during_first_advance_still_resumes</code> , <code>test_decide_after_cancel_is_rejected_and_not_hing_executes</code>	repository conditional updates; API 409s

Execution limits	<code>test_max_steps_exceeded ,</code> <code>test_max_run_time_exceeded ,</code> <code>test_approval_wait_not_counted_toward_run_time ,</code> <code>test_repeated_identical_call_terminates ,</code> <code>test_llm_backoff_that_exhausts_run_time_stops_run</code>	guardrails; lower-only overrides (422)
Control	<code>test_cancel_stops_run_at_next_tick ,</code> <code>test_extra_tool_calls_not_executed</code>	shutdown via lifespan; 500 without traceback

The fixes for the resume entry condition, the “run is waiting” decision condition and rejected-call refusal were each checked by reverting the fix and confirming the corresponding test fails.

11.1 Live run against a real model

make test-live with qwen2.5:7b (Ollama 0.34.0, local, auto-approve) on 2026-09-30, 4 min 44 s. The test only fails if the harness breaks; all five runs finished with a recorded reason and no internal errors. The model's behaviour is reported as observed; prompts and scenarios were not changed to improve it.

Scenario	Result	Tools as expected	What happened
status_check	completed, 2 steps	yes	One status call; correct summary of auth-service.
checkout_outage	completed, 3 steps	no	Parallel calls; the second call each time, including <code>create_incident</code> , was answered “not executed”. The model did not retry it, never checked payment-gateway, and falsely claimed in its answer that it had opened a SEV3 incident. No incident was created (§8.2).
runbook_lookup	completed, 6 steps	no	Asked how to mitigate connection-pool exhaustion, the model first called <code>get_service_status</code> with a non-existent service, used the error's list of known services to investigate checkout-api and payment-gateway, and proposed a SEV2 incident. The test auto-approves, so INC-000001 was created; with a human in the loop this is the call the gate exists to stop. The matrix would have said SEV1.
unknown_service	completed, 4 steps	no	Received “unknown service 'billing-service'; known services: ...”, then checked auth-service and checkout-api instead of reporting that billing-service does not exist. No incident.
maintenance_no_incident	completed, 3 steps	yes	Status (maintenance), then the maintenance-windows document; correctly advised against an incident.

Two of five tool sequences matched. The harness behaved as designed in every run: extra calls were answered, error observations carried enough detail for the model to change

course, and no side effect happened without an approval. The failures are model quality (task drift, ignoring the one-call-per-step rule, an invented claim), which is what a 7B local model is expected to show and why the approval gate does not rely on the model.

12. How to run

Command	What it does
<code>make install</code>	<code>uv sync</code>
<code>make demo</code> / <code>make demo-auto</code>	Scripted <code>checkout_outage</code> run; prompts for the approval / approves automatically; prints the trace
<code>make serve</code>	REST API on <code>http://127.0.0.1:8000</code> (OpenAPI at <code>/docs</code>)
<code>make check</code>	<code>ruff</code> , <code>mypy</code> , <code>pytest</code> with coverage
<code>make test-live</code>	<code>data/scenarios.json</code> against the configured real model
<code>make report</code>	Renders this document in Docker

Real model. Ollama: `LLM_PROVIDER=openai_compat` , `LLM_BASE_URL=http://localhost:11434/v1` , `LLM_API_KEY=ollama` , `LLM_MODEL=qwen2.5:7b` . OpenAI: default base URL, a real key and e.g. `gpt-4o-mini` .

Postman. `docs/postman_collection.json` (variables `baseUrl` , `runId` , `approvalId` ; folders Health, Runs, Approvals, Failure demos, Incidents). Test scripts capture the ids, so the folders run top to bottom against `make serve` .

Variable	Default	Variable	Default
<code>LLM_PROVIDER</code>	<code>scripted</code>	<code>MAX_STEPS</code> / <code>MAX_RUN_SECONDS</code>	12 / 120
<code>LLM_BASE_URL</code>	<code>https://api.openai.com/v1</code>	<code>MAX_PARSE_RETRIES</code>	2
<code>LLM_API_KEY</code>	<code>empty</code>	<code>MAX_CONSECUTIVE_TOOL_ERRORS</code>	3
<code>LLM_MODEL</code>	<code>gpt-4o-mini</code>	<code>REPEAT_CALL_LIMIT</code>	3
<code>LLM_TIMEOUT_SECONDS</code> / <code>LLM_MAX_RETRIES</code>	60 / 2	<code>TOOL_MAX_RETRIES</code> / <code>OBSERVATION_MAX_CHARS</code>	2 / 4000
<code>LLM_SEND_PARALLEL_TOOL_CALLS_FLAG</code>	<code>true</code>	<code>DATABASE_URL</code>	<code>sqlite+aiosqlite:///./harness.db</code>
<code>LOG_LEVEL</code> / <code>LOG_FORMAT</code>	INFO / json	<code>DATA_DIR</code>	<code>./data</code>
<code>MOCK_FAULTS</code>	<code>{}</code>	<code>ALLOW_FAULT_OVERRIDES</code> / <code>MOCK_SEED</code>	<code>true</code> / 42

13. Limitations

The limitations below are deliberate scope decisions (PLAN §2) unless stated otherwise.

- **Single process.** The per-run locks (`RunService._locks`) and the lock that allocates message and event sequence numbers (`Repository._append_lock`) live in one process, and the store is SQLite. Running two API processes against one database would allow two `advance()` calls on the same run.
- **A tick is not atomic across a crash.** Each repository write is its own transaction. A crash in the middle of a tick can leave, for example, a tool message without the matching run update. There is no resume-after-crash: a run left `RUNNING` by a crash stays so. A clean shutdown is handled: in-flight runs are marked `FAILED / internal_error` with message "shutdown".
- **No approval expiry.** A pending approval waits forever.
- **No approver authentication or roles.** `decided_by` is free text and the API has no authentication. This is also why per-run overrides may only lower limits.
- **Cancellation is cooperative.** A cancel takes effect at the next tick or at the loop's next save. A tool call already in flight finishes and is recorded, so an already-approved `create_incident` completes if the cancel lands while it is executing, and its events can appear after the cancel's `run_finished`.
- **One tool call per step.** Extra calls in one reply are answered "not executed"; independent read-only calls are not run in parallel, which costs steps.
- **Lock bookkeeping.** `RunService` keeps one lock entry per run for the life of the process.
- **Final answers are not checked against the trace.** The live run produced an answer claiming an incident had been opened when none was (§8.2). The harness records what actually happened, but it does not stop a model from stating otherwise.
- **Basic prompt-injection defence.** JSON wrapping, truncation and a system-prompt rule. There is no classifier or content filtering; the approval gate limits the consequence of an injection, not the injection itself.
- Also out of scope: token and cost budgets, semantic knowledge-base search, database migrations, OpenTelemetry export. The scripted provider replays one scenario for every objective.

14. Future improvements

- **Crash safety:** lease-based ownership of running runs (heartbeat + reclaim), and one transaction per tick (outbox for events) so a tick commits or not at all.
- **Horizontal scale:** Postgres with row locks (`SELECT ... FOR UPDATE SKIP LOCKED`) instead of in-process locks; a queue for background advances.
- **Approvals:** TTL with automatic rejection, approver authentication and roles, notification hooks.

- **Answer grounding:** check a final answer against the run's tool calls before accepting it (e.g. an incident id must come from a succeeded `create_incident`), and send a repair message otherwise.
- **Budgets:** token and cost limits per run using the `usage` already recorded on `llm_response` events.
- **Tools:** parallel execution of read-only calls; semantic KB search; per-tool rate limits and circuit breakers.
- **LLM:** multi-provider routing and fallbacks (e.g. LiteLLM [R9]) behind `LLMClient` .
- **Observability:** OpenTelemetry export of the same events as spans.
- **Operations:** Alembic migrations; lock cleanup when runs finish.

15. References

- [R1] Anthropic, *Building effective agents*. <https://www.anthropic.com/engineering/building-effective-agents>
- [R2] OpenAI Agents SDK, *Running agents*. https://openai.github.io/openai-agents-python/running_agents/
- [R3] OpenAI Agents SDK, *Human-in-the-loop*. https://openai.github.io/openai-agents-python/human_in_the_loop/
- [R4] LangGraph, *Human-in-the-loop / Interrupts*. <https://docs.langchain.com/oss/python/langgraph/human-in-the-loop>
- [R5] HumanLayer, *12-Factor Agents* (factors 5–9). <https://github.com/humanlayer/12-factor-agents>
- [R6] M. Brooker, *Timeouts, retries, and backoff with jitter*, Amazon Builders' Library. <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>
- [R7] Stripe API, *Idempotent requests*. https://docs.stripe.com/api/idempotent_requests
- [R8] OWASP, *Top 10 for LLM Applications 2025* (LLM01, LLM05, LLM06, LLM10). <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [R9] LiteLLM, *Router architecture (fallbacks / retries)*. https://docs.litellm.ai/docs/router_architecture